Artificial Models for Multimodal Creativity
Winter semester 2026/27 · Sound, symbols, sensors

Learning from Examples

LESSON 1 · HISTORY, PRINCIPLES AND A FIRST INSTRUMENT IN MAX

Prof. Dr. Alessandro Anatrini

16 October 2026

Hochschule für Musik und Theater Hamburg
Conservatorio di Musica “G. Nicolini”, Piacenza
Accademia di Belle Arti di Brera, Milano

Today

1	Audry's four myths about machine learning. AI, machine learning and deep learning; the history from Turing and cybernetics to the Transformer, reasoning models and agents	70 min
2	Learning a mapping from gesture to sound: data, a single weight computed by hand, the 86 parameters of the network you will train	50 min
3	Interactive machine learning, FluCoMa, and Exercise 1: you record positions of your phone together with sounds, and train a network that connects them	100 min
4	Discussion: the place of the artist, and what German courts decided about training data in 2025 and 2026	20 min

This semester

	Block	What the models learn	Tools
1	Learning from examples	a mapping from sensor data to synthesis parameters	FluCoMa in Max; phone sensors via OSC
2	Bodies as data	poses and gestures from a camera; gestures over time	MediaPipe and YOLO via OSC; FluCoMa; MuBu
3	Learning sequences	the next note, from a corpus of MIDI files	Markov chains in Max; Performance RNN via OSC
4	Latent spaces	a compressed representation of sound, from which sound can be generated	RAVE with `nn~`
5	Studio and critique	—	your pieces

In summer the same methods are applied to images and 3D in TouchDesigner and ComfyUI. Models that exist only in Python run in the background and send their output to Max over OSC; the scripts are provided.

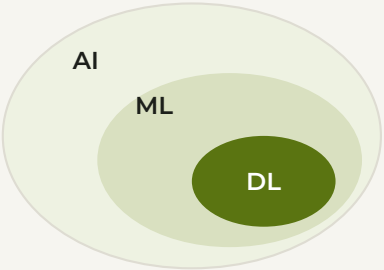
Four myths

Sofian Audry opens *Art in the Age of Machine Learning* (2021) with four common beliefs about machine learning, which the book sets aside:

1. Artificial intelligence, machine learning and deep learning are the same thing.
2. Machine learning is a new thing.
3. Artists are totally absent in machine-generated artworks.
4. Machine learning will give rise to superintelligence very soon.

Audry 2021, introduction; wording and numbering as in the book.

AI, machine learning, deep learning



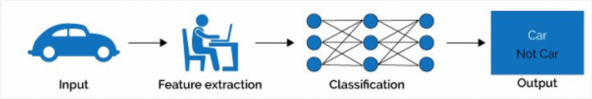
Artificial intelligence is the field named in the 1955 proposal for the Dartmouth workshop. It includes search, logic and planning, which need no learning: Deep Blue searched about 200 million chess positions per second and judged them with a function designed by people.

Machine learning covers methods that improve at a task from data. **Deep learning** is machine learning with networks of many layers.

A language model is a deep-learning model. An agent is a program built around one or more such models; it is not a different kind of learning. The three terms are nested, not synonyms, as Audry's first myth assumes.

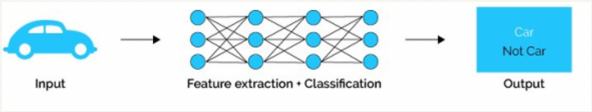
Who designs the features

CLASSICAL MACHINE LEARNING



The features are designed by people, often with knowledge of the domain (in audio: spectral centroid, loudness, pitch); a learning algorithm maps them to the output.

DEEP LEARNING



The network learns features and mapping together from raw or lightly processed data. This requires far more data, and the learned features are hard to interpret.

Turing's learning machine

"Instead of trying to produce a programme to simulate the adult mind, why not rather try to produce one which simulates the child's? If this were then subjected to an appropriate course of education one would obtain the adult brain."

ALAN TURING, COMPUTING MACHINERY AND INTELLIGENCE, 1950

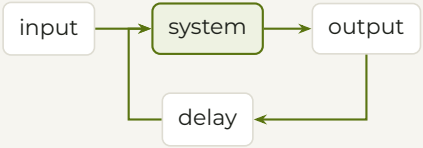
"An important feature of a learning machine is that its teacher will often be very largely ignorant of quite what is going on inside."

IBID., SECTION 7, LEARNING MACHINES

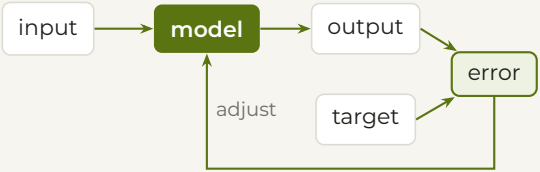
Both remarks describe the network of today's exercise: it is taught with examples instead of being programmed, and you will not be able to read from its 86 parameters why it produces a given sound.

Two kinds of feedback

FEEDBACK ON THE SIGNAL



FEEDBACK ON THE PARAMETERS



In a delay line with feedback the signal returns and changes itself: with N samples of delay the loop resonates at multiples of f_s/N. Max needs at least one signal vector of delay in a loop (`tapin~`/`tapout~`), `gen~` one sample (`history`).

In learning, what returns is the error, and what changes are the parameters of the model, step by step, until the error is small.

Cybernetics

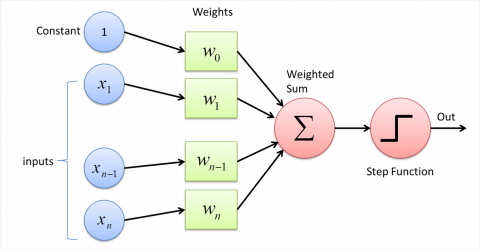
Wiener's *Cybernetics* (1948) is subtitled "control and communication in the animal and the machine": the same feedback mechanisms were studied in thermostats, anti-aircraft guns and nervous systems. The Macy conferences (1946–53) brought together McCulloch, Pitts, von Neumann, Bateson and Mead.

W. Ross Ashby's homeostat (1948) changed its internal settings at random whenever its outputs left a permitted range, and kept the first stable setting: it adapted to conditions nobody had programmed.

From the 1970s, second-order cybernetics (Heinz von Foerster) included the observer in the system being described: whoever builds or studies a system is part of it.

In the arts, *Cybernetic Serendipity*, curated by Jasia Reichardt at the ICA in London (1968), showed computer music, drawing machines and responsive sculpture. Gordon Pask's *Musicolour* (1953–57) turned a musician's playing into light and reduced its response when the musician repeated themselves, so that the player had to vary the material.

The formal neuron and the perceptron



Perceptron: each input *x* is multiplied by its weight *w*; the sum, plus a constant term, passes through a step function.

McCulloch and Pitts (1943) describe a neuron as a unit that fires when the sum of its binary inputs reaches a threshold, and show that networks of such units can compute logical functions. Their weights are fixed.

Hebb (1949) proposes that a connection between two cells is strengthened when they are active together.

Rosenblatt's perceptron (1958) learns its weights from labelled examples. The Mark I Perceptron at Cornell had 400 photocells as inputs; its weights were potentiometers turned by electric motors.

Adaptive and learning systems in music

Year	Work	Principle
1951	Ferranti Mark 1, Manchester: melodies programmed by Christopher Strachey	sound from a general-purpose computer
1953–57	Gordon Pask, *Musicolour*	a machine that responds to repetition
1957	Hiller & Isaacson, *Illiac Suite*	rules and Markov chains
from 1966	Roland Kayn, *Cybernetics I–III* and later works	self-regulating networks of signal processors
1968	Reich, *Pendulum Music*; Tudor, *Rainforest*	acoustic feedback
1969	Lucier, *I Am Sitting in a Room*	feedback through a room
from 1981	David Cope, *Experiments in Musical Intelligence*	recombination of a corpus of scores
from 1987	George Lewis, *Voyager*	an improvising program that listens to the performer
1990s	Nicolas Baginsky, *The Three Sirens*	robots with neural networks
from 2002	Agostino Di Scipio, *Audible Ecosystemics*	sound as the medium of self-regulation

Curtis Roads (1985) divides research on music and AI into composition, performance, music theory and signal processing.

From calculating machines to deep learning

	CALCULATION 1642–1939	CYBERNETICS 1940–1969	SYMBOLIC AI 1970–1993	STATISTICAL ML 1994–2011	DEEP LEARNING 2012–2016
Computing and AI	**1642** Pascal's calculating machine **1837** Babbage designs the Analytical Engine; Lovelace's notes, 1843	**1940** Turing's Bombe searches Enigma keys **1950** Turing proposes the imitation game **1956** Dartmouth workshop names the field **1966** Weizenbaum's ELIZA	**1974–80** first "AI winter" **1982** Japan's Fifth Generation project **1987–93** second "AI winter"	**1997** Deep Blue beats Kasparov **1998** Kismet, a social robot (MIT) **2005** Stanley wins the DARPA Grand Challenge **2011** Watson wins *Jeopardy!*; Siri	**2014** "Eugene Goostman" passes a Turing test: claim disputed **2016** AlphaGo beats Lee Sedol
Learning machines		**1943** McCulloch and Pitts: the formal neuron **1948** Wiener's *Cybernetics*; Ashby's homeostat **1949** Hebb's learning rule **1958** Rosenblatt's perceptron learns its weights **1959** Samuel coins "machine learning" **1969** Minsky and Papert, *Perceptrons*	**1986** backpropagation (Rumelhart, Hinton, Williams) **1989** LeCun's convolutional network reads digits	**1997** LSTM (Hochreiter, Schmidhuber) **2006** deep belief networks (Hinton) **2009** ImageNet: millions of labelled images	**2012** AlexNet: deep learning on GPUs **2013** DQN learns Atari games from pixels **2014** generative adversarial networks

Upper row: the timeline used in previous years (codebots.com), with dates and wording corrected.

Why neural networks came back

Through the 1990s and 2000s, support vector machines and other statistical methods gave better results than neural networks, which were hard to train with more than two or three layers. In 2012 a network with eight layers, AlexNet, won the ImageNet competition: 15.3% top-5 error, against 26.2% for the best other entry.

Data	1.2 million labelled training images in 1000 classes (ImageNet, from 2009)
Hardware	two consumer graphics cards (NVIDIA GTX 580); five to six days of training
Methods	ReLU activations, which train faster than sigmoids; dropout, which switches off random units during training against overfitting

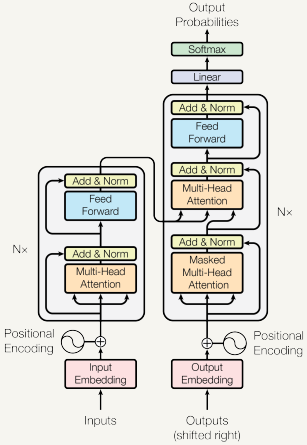
The learning procedure itself, backpropagation, dates from 1986. What changed after 2006 were the amounts of data and computation: in 2012 backpropagation was 26 years old, the perceptron 54.

Krizhevsky, Sutskever & Hinton 2012; Audry 2021, introduction.

From the Transformer to agents, 2017–2026

	PRE-TRAINING 2017–2019	SCALE 2020–2021	ASSISTANTS 2022–2023	REASONING, TOOLS 2024–2025	AGENTS AT WORK 2026
Language models	**2017** Transformer **2018** GPT, BERT: pre-training on raw text	**2020** GPT-3: 175 bn parameters; scaling laws; the task is given in the prompt	**2022** ChatGPT, tuned on human feedback **2023** GPT-4 reads images; Llama, open	**2024** o1 reasons before answering **2025** DeepSeek-R1, open; gold-medal level at the Mathematical Olympiad	**2026** GPT-5.5, Claude Opus 5: multi-step work over hours
Agents	**2017** AlphaGo Zero learns Go from self-play **2019** AlphaStar	**2020** retrieval-augmented generation	**2022** ReAct: reason, act, observe **2023** function calling; AutoGPT	**2024** computer use; MCP **2025** coding agents; orchestrators with subagents; A2A	**2026** multi-agent orchestration in IBM, Microsoft, Google platforms
Images, video	**2018** StyleGAN; *Edmond de Belamy* sold at Christie's	**2020** diffusion models **2021** CLIP, DALL·E	**2022** DALL·E 2, Midjourney, Stable Diffusion (open)	**2024** Sora: video from text **2025** Veo 3, Sora 2: video with synchronised sound; Genie 3: explorable worlds	
Music and sound	**2017** NSynth **2018** Music Transformer **2019** Herndon, *PROTO*	**2020** Jukebox; DDSP **2021** RAVE, real time	**2022** AudioLM **2023** MusicLM, MusicGen, Stable Audio	**2024** Suno v3, Udio: songs with vocals **2025** Magenta RealTime, open	**2026** Lyria 3; Stable Audio 3, Eleven Music v2 on licensed data; Suno v6
Rights and law	**2019** EU copyright directive: text and data mining exception	**2021** AI Act proposed; § 44b UrhG in force	**2023** Hollywood strikes: AI clauses in contracts	**2024** labels sue Suno, Udio; AI Act in force **2025** GEMA v. OpenAI; labels settle with Udio, Suno	**2026** GEMA v. Suno; UMG, Sony sue Suno again; AI Act deadlines moved

The Transformer



Vaswani et al. (2017) designed it for translation. The **encoder** (left) reads the source sentence; the **decoder** (right) writes the translation one token at a time, each new token conditioned on the ones already written.

Embedding: each token becomes a vector of 512 numbers. **Positional encoding** adds the position of the token, because attention by itself ignores order.

Multi-head attention: for every token the layer computes how much each other token matters to it, and takes a weighted mixture of their vectors. Eight heads do this in parallel and learn different relations. *Masked*: in the decoder a token sees only earlier tokens. The middle block of the decoder attends to the encoder's output.

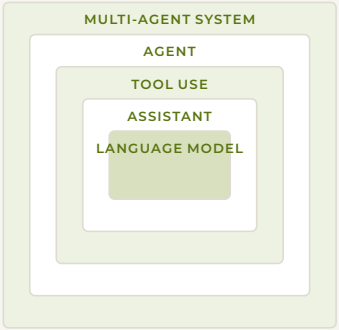
Feed forward: an MLP, the kind of network you train today, applied to each position. **Add & Norm**: the input of each block is added to its output, then rescaled. N = 6 layers per stack.

Linear, softmax: one probability for every token of the vocabulary; the next token is drawn from them.

GPT and today's language models keep only the decoder stack; BERT only the encoder. The Music Transformer (Huang et al., 2018) applies the same architecture to MIDI events.

Figure: Vaswani et al., "Attention is all you need", NeurIPS 2017, fig. 1.

From language model to agent



Language model. A Transformer trained to predict the next word fragment (token) of a text, on trillions of tokens.

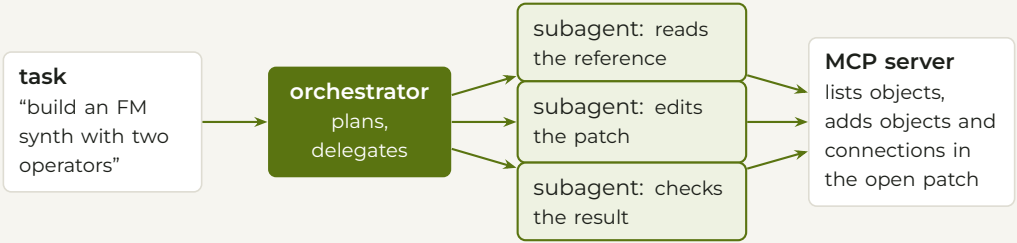
Assistant (2022). Further trained on example answers and on people's ratings of answers (reinforcement learning from human feedback).

Tool use (2023). The model writes a structured request, for instance a search or a calculation; a program executes it and returns the result into the text. MCP (2024) standardises how tools are connected.

Agent (2024–25). The model repeats a loop of planning, acting through tools and reading the result until the task is done: it edits files, runs code, operates a browser.

Multi-agent system (2025–26). An orchestrating agent divides a task, delegates parts to subagents with their own instructions and tools, and combines their results. A2A (2025) lets agents of different vendors exchange tasks.

An agent working in Max

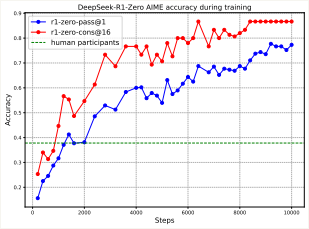


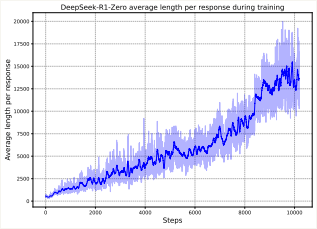
MaxMSP-MCP-Server (Tian et al., SMC 2025) connects a language-model agent to Max 9: the agent reads the objects and connections of an open patch, looks up documentation, explains patches and builds them. It also works with a single agent; the division above is one possible arrangement.

The authors name two difficulties: a patch is a graph, not a text, and few Max patches occur in the data the models were trained on.

Tian et al., SMC 2025; github.com/tiianhk/MaxMSP-MCP-Server.

Reasoning models



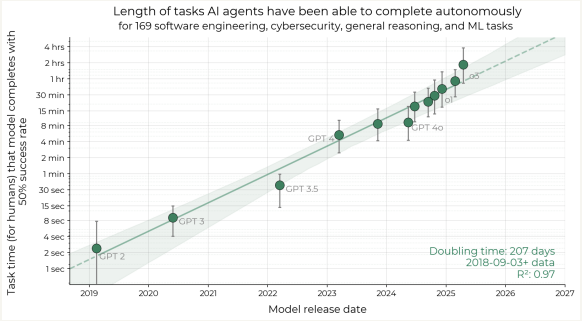


OpenAI o1 (September 2024) was the first model trained to write out intermediate steps before answering; its method was not published. DeepSeek-R1 (January 2025) documented one. DeepSeek-R1-Zero starts from a pre-trained language model and is trained only with reinforcement learning on mathematics and programming problems whose answers can be checked automatically. The reward checks only the final answer and its format; no example solutions are shown.

Over 10 000 training steps, accuracy on the AIME 2024 mathematics competition rises from 15.6% to 77.9% (left), and the average answer grows from a few hundred to more than 12 000 tokens (right). Nobody told the model to write longer: longer reasoning, with checks and restarts, is what the reward selected. A reasoning model therefore spends more computation on a harder question.

Figures: DeepSeek-AI, “DeepSeek-R1”, arXiv:2501.12948 (*Nature*, 2025), fig. 1.

Long tasks



METR timed professionals on 169 tasks in software engineering, cybersecurity, reasoning and machine learning, from seconds to many hours. A model's *time horizon* is the human duration of the tasks it completes in half of its attempts.

GPT-2 (2019): 2 seconds. GPT-4 (2023): about 5 minutes. o3 (April 2025): about 110 minutes. The horizon doubled every 207 days, about seven months.

For 80% success the horizons are about five times shorter. Tasks with unclear goals and feedback, closer to most real work, are solved less often than clean, automatically scored ones.

Figure: Kwa et al. (METR), "Measuring AI ability to complete long tasks", arXiv:2503.14499, fig. 1.

Narrow and general systems

Every model in this course is trained for one task and is reliable only on inputs similar to its training data. A network trained on your gestures produces arbitrary parameters for gestures it has not seen.

Large language and multimodal models handle a much wider range of tasks than earlier systems, and the METR curve shows the length of the tasks they complete growing steadily. Their capabilities still derive from training data, training objectives and subsequent tuning, and remain uneven: strong on mathematics and code, where answers can be checked and rewarded, weaker on tasks without such a check.

“Artificial general intelligence” and “superintelligence” name hypotheses about future systems, not descriptions of existing ones. Audry’s fourth myth is not that such systems are impossible, but that their arrival is imminent.

Audry 2021, introduction; Kwa et al. 2025.

Mapping in digital instruments

In an acoustic instrument the relation between gesture and sound is given by its physics. In a digital instrument it has to be designed. Hunt and Wanderley distinguish the basic cases:



In a clarinet, breath pressure and lip pressure together determine pitch, loudness and timbre: many-to-many. In experiments by Hunt and colleagues, players needed longer to learn such coupled mappings than one-to-one mappings, but after some practice played better with them. Coupled mappings are exactly the ones that are hard to write by hand.

Hunt & Wanderley, “Mapping performer parameters to synthesis engines”, *Organised Sound* 7(2), 2002.

A gesture controls a sound

The accelerometer of a phone sends three numbers about fifty times a second. A synthesiser has six parameters: carrier frequency, ratio, modulation index, cutoff, resonance, gain. The instrument needs a function that turns three numbers into six.

WRITTEN BY HAND

With `scale` objects, one input per parameter, the mapping is quick to write and easy to predict. As soon as each parameter should depend on all three inputs, the formulas become hard to find and harder to adjust by ear.

LEARNED FROM EXAMPLES

You hold the phone in a position, set the six parameters to a sound you want there, and record the pair. After eight or ten pairs, a neural network computes a function that passes through all of them and fills the space between.

Four ways of learning

	Training data	In this course	In training a chat assistant
Supervised	inputs with the desired outputs	phone position → synthesis parameters (block 1)	example answers written by people
Unsupervised	inputs only; the method finds structure	sounds arranged on a map by similarity (block 2)	—
Self-supervised	the target is taken from the data itself	next note of a melody (block 3); a sound reconstructed from a code (block 4)	next token of a text: pre-training
Reinforcement	actions, and a reward for their outcome	—	ratings of answers; checked solutions for reasoning

A chat assistant goes through three of the four: self-supervised pre-training on text, supervised tuning on example answers, reinforcement learning on ratings and on problems with checkable solutions.

The data of Exercise 1

		In Exercise 1
Example	one input with its target	one position of the phone, held still, with one sound
Features	the numbers that describe the input	acceleration on x, y and z
Target	the output the model should give for it	six synthesis parameters, each between 0 and 1
Dataset	all examples	eight to twelve positions
Outlier	an example far from the others	a position recorded while the phone was still moving

At rest the accelerometer measures gravity, hence the tilt of the phone, but not its rotation around the vertical axis. The compass heading, as a fourth feature, would add that rotation.

Tensors

In machine learning, *tensor* means an array of numbers with any number of axes. Its **shape** gives the length of each axis.

By convention the first axis counts the examples. The order of the other axes depends on the library: PyTorch stores images as (batch, channels, height, width), TensorFlow as (batch, height, width, channels).

Shape	Content
`()`	one number: a MIDI velocity, or the acceleration along one axis
`(3,)`	one accelerometer reading: acceleration along x, y and z
`(10, 3)`	the inputs of Exercise 1
`(10, 6)`	the targets of Exercise 1
`(20, 100, 3)`	20 gestures, 100 readings each
`(16, 2, 48000)`	16 stereo clips of 1 s at 48 kHz
`(128, 3, 256, 256)`	128 colour images

Regression and classification

REGRESSION

The target is a quantity: a cutoff frequency, a gain, a position. The loss measures the distance between output and target, as in the single-weight example.

CLASSIFICATION

The target is one of a fixed set of categories: which sample to play, which section of a piece, which instrument is sounding. The model outputs one probability per category.

Pitch looks like a quantity but is usually treated as a category. As a number, a prediction of 60.5 falls between C and C#, and an output one octave off is counted as twelve times worse than one a semitone off, though musically it may be the better guess. Melody models therefore output 128 probabilities, one per MIDI pitch.

Discriminative and generative models

DISCRIMINATIVE

Computes an output for an input: a class, a value, a list of parameters. The same input always gives the same output. The network of Exercise 1 is of this kind; so is a classifier that recognises a hand pose.

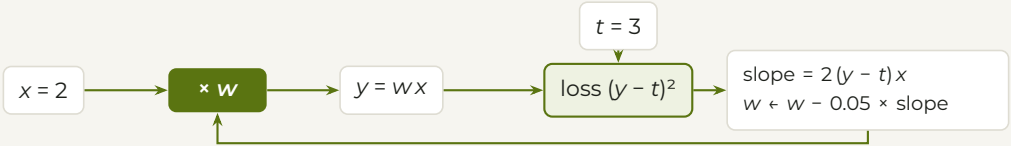
GENERATIVE

Learns how likely each possible output is and draws one at random. A melody model gives, for each of the 128 MIDI pitches, the probability that it comes next; the next note is drawn from that distribution.

The *temperature* reshapes the distribution before drawing. At low temperature the most likely pitch is chosen almost every time, and melodies repeat; at high temperature unlikely pitches become frequent.

A generative model can only produce what is probable under its training data. Which music it can generate is decided by which music it was trained on.

One weight, one example



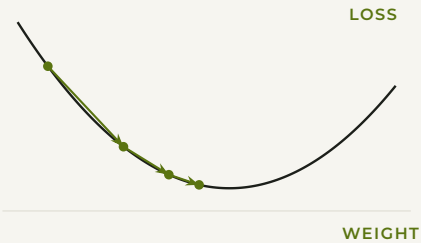
The correct weight is 1.5. We start from $w = 0.5$, so $y = 1$ and the loss is 4. Squaring keeps the loss positive and makes large errors count more.

The slope says in which direction, and how steeply, the loss changes when w changes. Each step moves w against the slope; the factor 0.05 is the *learning rate*.

At every step the distance from the correct weight shrinks by the same factor, 0.6. After ten steps the output is 2.988.

step	w	y	loss	slope
0	0.500	1.000	4.000	−8.00
1	0.900	1.800	1.440	−4.80
2	1.140	2.280	0.518	−2.88
3	1.284	2.568	0.187	−1.73
10	1.494	2.988	0.000	−0.05

The learning rate



rate	steps until *y* is within 0.01 of 3
0.01	64
0.05	11
0.125	1
0.25	never: *w* jumps between 0.5 and 2.5
0.30	never: every step lands further away

In the example the distance from the correct weight is multiplied at each step by 1 − 8 × rate. With many weights the loss is a surface over thousands of dimensions and the best rate is found by trying.

Momentum adds a fraction of the previous step to the current one, which speeds up progress along long, shallow slopes.

Scaling the inputs

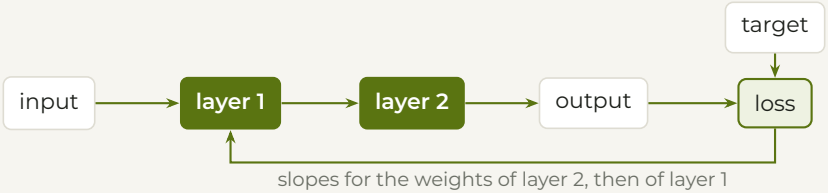
In the single-weight example the gap shrinks by $1 - 2x^2 \times$ rate per step: the input enters the size of each step. With $x = 20$ instead of 2, the rate 0.05 that converged in 11 steps makes training diverge.

Acceleration measured in *g* lies roughly between −2 and 2; a compass heading lies between 0 and 360 degrees. Fed unscaled to a network, the heading dominates the weighted sums and the training steps, and the acceleration hardly counts.

Method	Each feature is mapped to	FluCoMa
normalisation	the range 0 to 1, from its minimum and maximum	`fluid.normalize~`
standardisation	mean 0 and standard deviation 1	`fluid.standardize~`
robust scaling	median 0, scaled by the spread between two percentiles; little affected by outliers	`fluid.robustscale~`

The minimum, maximum or mean are computed once, on the training examples, and stored. During performance every new reading is scaled with the stored values.

Many weights: backpropagation

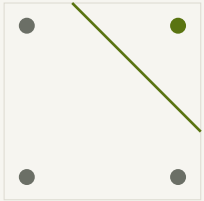


In a network every weight is treated as in the single-weight example: it gets its own slope and moves against it.

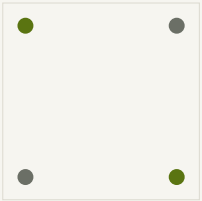
Backpropagation computes all slopes in one pass backwards from the loss: the slope of a weight in layer 1 depends on how its output affects layer 2 and how layer 2 affects the loss, and these intermediate results are reused. Rumelhart, Hinton and Williams made the method known in 1986.

Updates are made after each small group of examples (*batch*); one pass through all the data is an *epoch*.

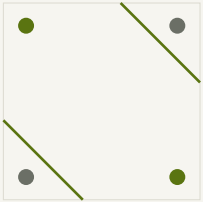
What a single layer cannot do



AND



XOR

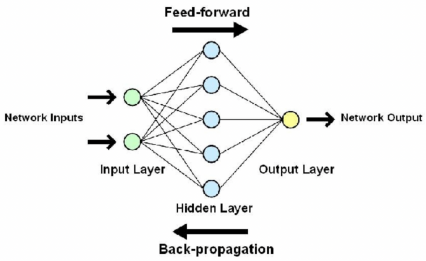


XOR WITH A HIDDEN LAYER

Two inputs, each 0 or 1; the green points have output 1. A perceptron separates the two classes with one straight line. That works for AND; for XOR (output 1 when exactly one input is 1) no such line exists.

With a hidden layer of two units, each draws one line and the output unit combines them. Minsky and Papert proved the limit of the single layer in 1969; their book is counted among the causes of the first decline of neural-network research.

Hidden layers

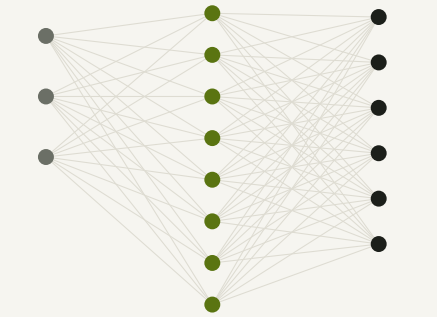


A multilayer perceptron (MLP) has one or more hidden layers. Each unit applies an *activation function* to a weighted sum of the previous layer: sigmoid, tanh, or ReLU, which sets negative values to 0.

The activation must be non-linear: a weighted sum of weighted sums is again a weighted sum, and without it the hidden layer adds nothing.

With one hidden layer and enough units, an MLP can approximate any continuous function on a bounded range of inputs (Cybenko 1989; Hornik 1991), but the theorem does not say how many units, nor that training finds the weights.

Counting the parameters



Each connection has a weight; each hidden and output unit has one more constant term, the bias: (3 × 8 + 8) + (8 × 6 + 6) = 32 + 54 = **86 parameters**.

A layer of 1000 units connected to every pixel of a 224 × 224 colour image needs 224 × 224 × 3 × 1000, about 150 million weights.

A convolutional layer slides the same small filter over the image: 64 filters of 3 × 3 pixels × 3 colours need 64 × 27 + 64 = 1792 parameters. This saving is one reason why images and audio are processed with convolutional layers.

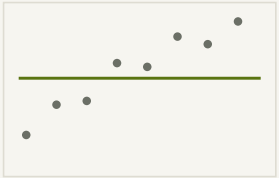
Training, validation and test data



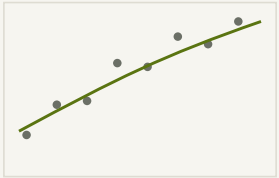
A model is useful if it works on inputs it has not seen. With a large dataset, part of it is set aside to check this. The **training** data adjust the weights. The **validation** data are used during development, to compare settings and to stop training when the error on them stops falling. The **test** data are used once, at the end.

With ten examples recorded by hand, setting aside two or four of them leaves gaps in the instrument. In interactive work all examples are used for training, and the test is playing: you move through positions you did not record and listen.

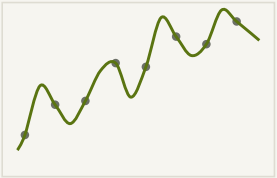
Underfitting and overfitting



UNDERFITTING



APPROPRIATE

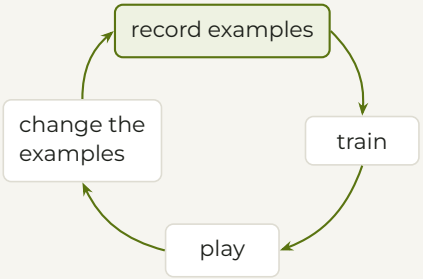


OVERFITTING

A model that is too simple misses the shape of the data and is wrong on old and new inputs alike. A model with too much freedom also fits the noise: its error on the training data is near zero, its error on new data large.

For an instrument, passing exactly through every recorded example is often intended. What decides whether it is overfitted is what it does between the examples: smooth transitions, or sudden jumps and swings.

Interactive machine learning



Rebecca Fiebrink's Wekinator (2009) receives sensor data over OSC, records examples, trains in a few seconds and sends its outputs over OSC to any program. It offers classifiers (k-nearest neighbours, decision trees, AdaBoost, support vector machines), neural networks for continuous outputs, and dynamic time warping for gestures in time.

In Fiebrink's studies with composers, the usual response to an instrument that did not behave as wanted was to add, delete or re-record examples, much more often than to change the algorithm or its settings.

Fiebrink, Trueman & Cook, NIME 2009; Fiebrink, PhD thesis, Princeton, 2011.

Small data, fast training

	A large language model	The network of Exercise 1
Data	trillions of tokens of text written by others	ten examples you record
Parameters	hundreds of billions	86
Training	months on thousands of processors	under a second on a laptop
Evaluation	scores on held-out test sets	you play it

Laetitia Sonami has played her *Spring Spyre* through networks trained in this way since 2012. Fiebrink and Sonami describe how she retrained the instrument over eight years, and how training became part of composing for it.

Fiebrink & Sonami, “Reflections on eight years of instrument creation with machine learning”, NIME 2020.

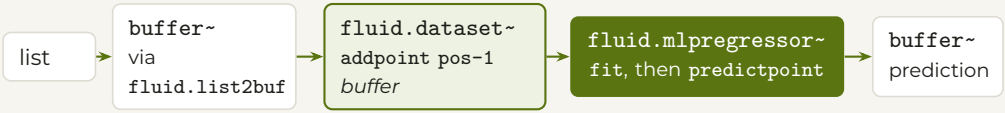
FluCoMa

Fluid Corpus Manipulation is a set of more than eighty objects for Max, Pd and SuperCollider, developed in a research project at the University of Huddersfield led by Pierre Alexandre Tremblay and funded by the European Research Council. It is free; since 2024 its hosting is supported by the Conservatorio della Svizzera italiana.

Analysis	descriptors: `fluid.pitch~`, `fluid.loudness~`, `fluid.mfcc~`, `fluid.spectralshape~`; segmentation: `fluid.onsetslice~`, `fluid.noveltyslice~`
Decomposition	`fluid.hpss~` (harmonic and percussive parts), `fluid.sines~`, `fluid.bufnmf~` (non-negative matrix factorisation)
Data	`fluid.dataset~`, `fluid.labelset~`, `fluid.bufstats~`, `fluid.plotter`
Scaling, reduction	`fluid.normalize~`, `fluid.standardize~`, `fluid.pca~`, `fluid.umap~`, `fluid.mds~`
Learning	`fluid.kdtree~`, `fluid.kmeans~`, `fluid.knnclassifier~`, `fluid.knnregressor~`, `fluid.mlpclassifier~`, `fluid.mlpregressor~`

FluCoMa was designed for collections of sounds. Today we use only the learning objects; blocks 2 and 4 use the rest.

How data move through FluCoMa



FluCoMa objects read and write `buffer~` objects, not Max lists. A point (the features of one example, or its targets) is a short buffer with one value per frame; `fluid.list2buf` and `fluid.buf2list` convert between lists and buffers.

A `fluid.dataset~` stores points under identifiers you choose (`pos-1`, `pos-2`, …). Supervised learning uses two datasets with the same identifiers, one for inputs and one for targets. A `fluid.labelset~` stores a category name per identifier, for the classifiers.

Because everything passes through buffers, the same objects can learn from descriptors computed on audio files and from control data.

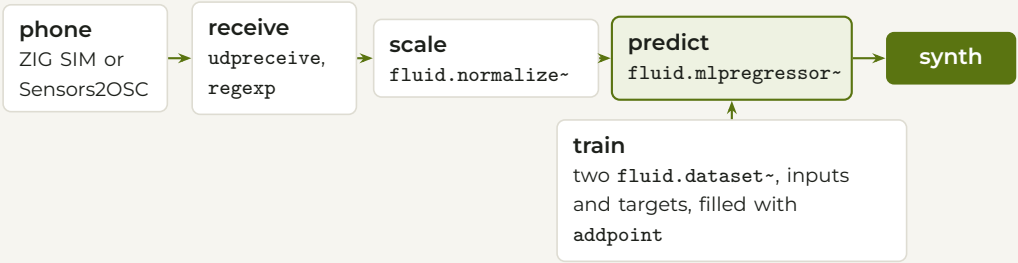
The MLP in FluCoMa

The settings are chosen before training. In Max they are attributes, written in lowercase.

Attribute	Default	Meaning
`hiddenlayers`	`3 3`	units per hidden layer; `8` means one layer of 8
`activation`	`2`	0 identity, 1 sigmoid, 2 ReLU, 3 tanh
`outputactivation`	`0`	activation of the output layer
`learnrate`	`0.01`	the learning rate
`momentum`	`0.9`	fraction of the previous step added
`maxiter`	`1000`	epochs per `fit` message
`batchsize`	`50`	examples per update
`validation`	`0.2`	fraction of examples held out

With `validation 0.2`, every `fit` removes a different random fifth of your examples from training and may stop early. For a hand-recorded dataset set it to 0. Changing `activation` or `hiddenlayers` erases what the network has learned.

Exercise 1: tilt and timbre



The patch `ex1-tilt-timbre.maxpat` contains the synthesiser, `ex1.fm-synth`: two-operator FM followed by a resonant low-pass filter, six parameters between 0 and 1. Record eight to ten positions with their sounds, train, then play the space between them. Without a phone, three sliders take the place of x, y and z; the rest of the patch stays the same.

Steps

1. Install FluCoMa from the Max Package Manager. On the phone, ZIG SIM with GRAVITY enabled (its ACCEL excludes gravity and does not measure tilt), or Sensors2OSC with the accelerometer.
2. Connect the phone and the laptop to the same network, port 8000. Print what arrives at `udpreceive`: the patch accepts addresses ending in `gravity` or `accelerometer`.
3. For each position: hold the phone still, set the six parameters, convert input and target lists with `fluid.list2buf` and send `addpoint pos-`*n buffer* to each dataset.
4. Send `fittransform` *inputs scaled* to `fluid.normalize~`: it stores the ranges and writes a scaled copy of the input dataset.
5. On `fluid.mlpregressor~ @hiddenlayers 8 @activation 3 @outputactivation 1 @learnrate 0.1 @batchsize 1 @validation 0`, send `fit` *scaled targets* several times; it answers with the error after each run.
6. While playing: scale each reading with `transformpoint`, send `predictpoint` *in out*, convert the result with `fluid.buf2list` and send it to the synthesiser.

EXERCISE 1

Variations

Change	Listen for
3 examples, then 20	how much of the space between them is predictable
two sounds for the same position	the network averages them
unscaled compass heading as fourth input	the tilt stops counting
`hiddenlayers 2`, then `32 32`	smooth, then abrupt transitions
`activation 1` (sigmoid), then `2` (ReLU)	curves, then straight segments
`fluid.knnregressor~`, same data	steps between neighbours
`fluid.knnclassifier~` with a labelset	each position selects a sound file

Write down one thing the instrument does that none of your examples specified: the discussion starts from these notes.

EXERCISE 1

Pd and Plugdata

OSC in Pd: `netreceive -u -b 8000` → `oscparse` → `list trim` → `route` with the address.

In Pd vanilla, FluCoMa works with the same object names and messages as in Max.

In Plugdata, use Wekinator: it is a separate program and needs no externals. It receives inputs as `/wek/inputs` on port 6448 and sends outputs as `/wek/outputs` to port 12000.

The Plugdata patch sends the three accelerometer values to Wekinator and receives six values back.

Data, model, training

Data
you chose the features (tilt, not rotation), the positions and their sounds

Model
you chose the network: one hidden layer of 8 units, the activation

Training
you chose the learning rate, how many times to send `fit`, when to stop

Audry's third myth is that artists are absent from works made with machine learning. Her book is organised around these three materials.

The sounds between your positions were chosen by nobody. They follow from your examples and from the structure of the network, and you heard them only after training. You can keep them, or change the examples until the network produces something else.

With Suno or Udio the situation is reversed: the data are millions of recordings by other musicians, model and training belong to the company, and your choices are limited to the prompt and to the selection among the results.

Audry 2021, introduction and parts I–III.

Training data before German courts

GEMA v. OpenAI LG München I, 11 Nov. 2025	The lyrics of nine songs, among them *Atemlos* and *Über den Wolken*, were memorised by the models and reproduced in answers. The court treated memorisation in the model as a reproduction (§ 16 UrhG) and held that the text and data mining exception (§ 44b UrhG) does not cover it.
GEMA v. Suno LG München I, 31 July 2026	Six songs, among them *Atemlos durch die Nacht* and *Daddy Cool*. The same reasoning, applied to recordings; Suno is also held responsible for its outputs.

Both judgments are at first instance, and both companies announced appeals. In the USA, Universal and Sony sued Suno a second time in September 2026, after Suno released v6, a model built with Warner, BMG and Believe.

In 2026 Stability AI (Stable Audio 3) and ElevenLabs (Music v2) released music models trained, by their own account, only on licensed recordings.

DISCUSSION

The four myths and Exercise 1

Myth	In the network you trained
1 · AI, ML and DL are the same	It is machine learning with a neural network, but with one hidden layer it is not deep. The features (three accelerations) were chosen by you, as in classical machine learning.
2 · Machine learning is new	The formal neuron dates from 1943, learned weights from 1958, backpropagation from 1986. FluCoMa and Wekinator make them playable in real time.
3 · Artists are absent	You chose the data, the model and the training; the network chose the sounds between your examples.
4 · Superintelligence very soon	86 parameters, one task, reliable only near its examples. Large models extend the range of tasks; METR measures how far, on tasks whose results can be checked.

Questions

1. Your network produced sounds for positions you never recorded. Are they part of your work in the same way as the recorded ones?
2. Would it be the same instrument if a classmate had recorded the examples? If they came from a public dataset of gestures?
3. The Munich court treated what a model has memorised as a copy. What would count as a copy in a network that has learned your examples?

DISCUSSION

Before the next block

- Read the introduction of Audry, *Art in the Age of Machine Learning*.
- Finish Exercise 1 and record one minute of playing it.
- Bring a laptop with a working webcam: block 2 uses MediaPipe for hands and body, and YOLO for people and objects in the room.
- Describe in a few lines three gestures that have a function in a piece of yours.

References

Audry, S. (2021). *Art in the Age of Machine Learning*. MIT Press.
Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2.
Fiebrink, R. (2011). *Real-time Human Interaction with Supervised Learning Algorithms for Music Composition and Performance*. PhD thesis, Princeton.
Fiebrink, R., & Sonami, L. (2020). Reflections on eight years of instrument creation with machine learning. *NIME*.
Fiebrink, R., Trueman, D., & Cook, P. R. (2009). A meta-instrument for interactive, on-the-fly machine learning. *NIME*.
DeepSeek-AI (2025). DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning. *Nature*, 645; arXiv:2501.12948.
Huang, C.-Z. A., et al. (2018). Music Transformer. arXiv:1809.04281.
Hunt, A., & Wanderley, M. M. (2002). Mapping performer parameters to synthesis engines. *Organised Sound*, 7(2).
Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS*.
Kwa, T., et al. (2025). Measuring AI ability to complete long tasks. arXiv:2503.14499.
McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5.
Minsky, M., & Papert, S. (1969). *Perceptrons*. MIT Press.
Roads, C. (1985). Research in music and artificial intelligence. *ACM Computing Surveys*, 17(2), 163–190.
Rosenblatt, F. (1958). The perceptron. *Psychological Review*, 65(6).
Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323.
Tian, H., et al. (2025). Agentic Max/MSP programming with LLMs and MCP. *SMC*, late-breaking demo.
Turing, A. M. (1950). Computing machinery and intelligence. *Mind*, 59(236).
Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.
Wiener, N. (1948). *Cybernetics*. MIT Press.

FluCoMa **Wekinator** **MaxMSP-MCP-Server** **Sensors2OSC**